

Informe - Milestone 3

1. Objetivo y punto de partida

El agente parte del M1 (calculator, file_reader, text_search) y se le agregan las herramientas de sala de escape: look, examine, take, use y go. El baseline ya tenía loop ReAct, registro dinámico de tools, memoria de historial acotada (max_history_messages=50), límite anti-loop (max_iterations=10) y manejo de errores sin romper la ejecución. Faltaba: prompt especializado, memoria de mapa/inventario, evaluación automática y clasificación de errores.

2. Mejoras más importantes

2.1 Prompt especializado en sala de escape

El primer salto de calidad fue reemplazar el prompt genérico por uno con reglas de dominio: copiar ids exactos, no repetir llamadas que ya dieron el mismo resultado, y sobre todo la secuencia examinar → tomar → usar antes de intentar abrir algo. Con esto, la sala de escape testeada inicialmente, color-locks, pasó de quedar trabado examinando cofres en loop a resolverse completo.

2.2 Memoria de navegación en el prompt

En apartment-keys se detectaron errores de navegación, como intentos de moverse por salidas inexistentes y de usar un ítem sobre un objetivo no visible en la sala actual. La causa era falta de memoria de mapa, no de razonamiento. Agregar al prompt la obligación de llevar un mapa mental (sala actual, salidas conocidas, camino recorrido) redujo el recorrido de 18 a 11 pasos.

2.3 Reconocer el objetivo cumplido

Un patrón recurrente fue que el agente lograba el objetivo, pero seguía actuando hasta agotar iteraciones (post_goal_overrun). La solución fue una regla de corte explícita: en cuanto una tool confirma que la puerta principal se abrió, responder de inmediato sin llamar a ninguna otra herramienta.

2.4 Política anti-loop de examine

En library-search, con ocho libros para revisar, el agente entraba en loop de look apenas el historial se llenaba de texto largo. Se agregó la regla de continuar con el siguiente id pendiente de la lista ya revelada en vez de volver a mirar la sala, lo que permitió examinar los ocho libros ordenadamente y resolver el escenario.

2.5 De memoria pasiva a memoria activa + guardrails

Agregar memoria estructurada como texto extra en el prompt no funcionó: empeoró el success rate (de 0.875 a 0.75) porque el agente podía ignorar hechos ya conocidos. El cambio efectivo fue convertir la memoria en guardrails activos que bloquean la ejecución real de una tool antes de llamarla, por ejemplo, go se bloquea si la dirección no existe desde la sala actual, y use se bloquea si el ítem no está en el inventario observado. Esto redujo office-sequence de 31 a 20 pasos.

2.6 Enseñar a usar la memoria, no solo dársela

Sobre vault-combination, el escenario más persistente en fallar, se comprobó que no alcanza con tener memoria estructurada: el prompt también debe indicar cómo priorizarla. prompt/04_PROMPT, con instrucciones explícitas de revisar sala actual, inventario, objetos revelados no tomados y salidas conocidas antes de cada acción, llevó el resultado de 0/3 a 3/3 corridas exitosas.

2.7 Evaluación automatizada y metodología

Se construyó eval/run.py para evaluar con check_goal (no con el texto del agente) y clasificar errores automáticamente (navigation_error, repeated_action, post_goal_overrun, max_iterations). También se detectó que una sola corrida por variante era insuficiente por el no determinismo del LLM, por lo que se agregó soporte de múltiples corridas (--runs N) y comparación por promedios. Se sumó además una rúbrica cualitativa (0 a 3) basada en señales observables de la traza.

3. Resumen de mejoras y resultados

Etapa	Cambio clave	Resultado
Prompt especializado	Reglas de sala de escape (examinar → tomar → usar)	color-locks pasó de fallar a resolverse
Reglas de navegación	Mapa mental de salas/salidas en el prompt	apartment-keys: 18 → 11 pasos
Anti-loop de examine	No repetir look si hay ids pendientes por examinar	library-search resuelto sin loops
Corte al cumplir objetivo	Frenar apenas la puerta se abre	Elimina post_goal_ouerrun
Memoria activa (guardrails)	Bloquear go/use si contradicen la memoria	office-sequence: 31 → 20 pasos
O4_PROMPT + memoria	Prompt que explica cómo priorizar la memoria	vault-combination: 0/3 → 3/3

4. Conclusión

Cada mejora nació de un fallo concreto observado en una traza. El prompt engineering resolvió buena parte de los problemas de secuencia y de reconocimiento de objetivo, pero mostró un techo en navegación e inventario, donde la solución efectiva no fue más texto sino memoria activa con guardrails, combinada con instrucciones explícitas sobre cómo usarla.